

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 4**



VIEWMODEL DAN DEBUGGING

Oleh:

Dina Izzati Elfadheya NIM. 2310817120001

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
MEI 2025**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 4

Laporan Praktikum Pemrograman Mobile Modul 4: Viewmodel dan Debugging ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Dina Izzati Elfadheya

NIM : 2310817120001

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Zulfa Auliya Akbar
NIM. 2210817210026

Muti`a Maulida S.Kom M.T.I
NIP. 19881027 201903 20 13

DAFTAR ISI

LEMBAR PENGESAHAN	2
DAFTAR ISI	3
DAFTAR GAMBAR	4
DAFTAR TABEL.....	5
SOAL 1	6
A. Source Code	7
B. Output Program.....	27
C. Pembahasan.....	31
D. Tautan Git.....	45
SOAL 2	46
A. Jawaban.....	46

DAFTAR GAMBAR

Gambar 1 Halaman Home.....	27
Gambar 2 Halaman Detail.....	28
Gambar 3 Link Netflix.....	29

DAFTAR TABEL

Tabel 1 Source Code Jawaban MainActivity.kt	7
Tabel 2 Source Code Jawaban Drama.kt.....	7
Tabel 3 Source Code Jawaban HomeFragment.kt	10
Tabel 4 Source Code Jawaban DetailFragment.kt	11
Tabel 5 Source Code Jawaban DramaAdapter.kt.....	13
Tabel 6 Source Code Jawaban DramaViewModel.kt.....	14
Tabel 7 Source Code Jawaban DramaViewModelFactory.kt.....	15
Tabel 8 Source Code Jawaban Activity_Main.xml	15
Tabel 9 Source Code Jawaban fragment_home.kt	16
Tabel 10 Source Code Jawaban fragment_detail.kt	19
Tabel 11 Source Code Jawaban item_list.kt.....	21
Tabel 12 Source Code Jawaban string.kt	23
Tabel 13 Source Code Jawaban build. gradle.kts.....	25
Tabel 14 Source Code Jawaban AndoidManifest.kt	26

SOAL 1

Lanjutkan aplikasi Android berbasis XML dan Jetpack Compose yang sudah dibuat pada Modul 3 dengan menambahkan modifikasi sesuai ketentuan berikut:

- a. Buatlah sebuah ViewModel untuk menyimpan dan mengelola data dari list item. Data tidak boleh disimpan langsung di dalam Fragment atau Activity.
- b. Gunakan ViewModelFactory dalam pembuatan ViewModel
- c. Gunakan StateFlow untuk mengelola event onClick dan data list item dari ViewModel ke Fragment
- d. gunakan logging untuk event berikut:
 - a. Log saat data item masuk ke dalam list
 - b. Log saat tombol Detail dan tombol Explicit Intent ditekan
 - c. Log data dari list yang dipilih ketika berpindah ke halaman Detail
- e. Gunakan tool Debugger di Android Studio untuk melakukan debugging pada aplikasi. Cari setidaknya satu breakpoint yang relevan dengan aplikasi. Lalu, gunakan fitur Step Into, Step Over, dan Step Out. Setelah itu, jelaskan fungsi Debugger, cara menggunakan Debugger, serta fitur Step Into, Step Over, dan Step Out

A. Source Code

1. MainActivity.kt

```
1 package com.example.viewmodel
2
3 import android.os.Bundle
4 import androidx.appcompat.app.AppCompatActivity
5 import com.example.viewmodel.R
6
7 class MainActivity : AppCompatActivity() {
8     override fun onCreate(savedInstanceState:
Bundle?) {
9         super.onCreate(savedInstanceState)
10        setContentView(R.layout.activity_main)
11
12        val fragmentManager =
supportFragmentManager
13        val homeFragment = HomeFragment()
14        val fragment =
fragmentManager.findFragmentByTag (HomeFragment::
class.java.simpleName)
15        if (fragment !is HomeFragment) {
16            fragmentManager.beginTransaction()
17                .add(R.id.frame_container,
homeFragment,
HomeFragment::class.java.simpleName)
18                .commit()
19        }
20    }
21 }
```

Tabel 1 Source Code Jawaban MainActivity.kt

2. Drama.kt

```
1 package com.example.recyclerview
2
3 import android.os.Parcelable
4 import kotlinx.parcelize.Parcelize
5
6 @Parcelize
7 data class Drama(
8     val title: String,
9     val link: String,
10    val photo: String,
11    val plot: String,
12    val year: String,
13    val cast: String
14 ) : Parcelable
```

Tabel 2 Source Code Jawaban Drama.kt

3. HomeFragment.kt

```
1 package com.example.viewmodel
2
3 import android.content.Intent
4 import android.net.Uri
5 import android.os.Bundle
6 import android.view.LayoutInflater
7 import android.view.View
8 import android.view.ViewGroup
9 import androidx.fragment.app.Fragment
10 import androidx.fragment.app.viewModels
11 import androidx.lifecycle.lifecycleScope
12 import
13 androidx.recyclerview.widget.LinearLayoutManager
14 import
15 com.example.viewmodel.databinding.FragmentHomeBi
16 nding
17 import kotlinx.coroutines.flow.collectLatest
18 import kotlinx.coroutines.launch
19
20 class HomeFragment : Fragment() {
21     private var _binding: FragmentHomeBinding? =
22     null
23     private val binding get() = _binding!!
24     private val viewModel: DramaViewModel by
25     viewModels {
26
27     DramaViewModelFactory(requireContext().resources
28     )
29     }
30     override fun onCreateView(
31         inflater: LayoutInflater, container:
32         ViewGroup?,
33         savedInstanceState: Bundle?
34     ): View {
35         _binding =
36         FragmentHomeBinding.inflate(inflater, container,
37         false)
38         return binding.root
39     }
40     override fun onViewCreated(view: View,
41         savedInstanceState: Bundle?) {
```

```

35         binding.rvDrama.layoutManager =
LinearLayoutManager(requireContext())
36         binding.rvDrama.setHasFixedSize(true)
37
38         val adapter = DramaAdapter(
39             listDrama = emptyList(),
40             onWikiClick = { link ->
41                 val uri = Uri.parse(link)
42                 startActivity(Intent(Intent.ACTION_VIEW, uri))
43             },
44             onDetailClick = { drama ->
45                 viewModel.onDramaClicked(drama)
46             }
47         )
48
49         binding.rvDrama.adapter = adapter
50
51         lifecycleScope.launch {
52             viewModel.dramaList.collectLatest {
dramas ->
53                 adapter.updateData(dramas)
54             }
55         }
56
57         lifecycleScope.launch {
58             viewModel.eventClick.collectLatest {
drama ->
59                 android.util.Log.d(
60                     "HomeFragment",
61                     "Pindah ke Detail dengan
data: ${drama.title}, ${drama.year},
${drama.cast}"
62                 )
63
64                 val detailFragment =
DetailFragment().apply {
65                     arguments = Bundle().apply {
66                         putString("EXTRA_TITLE",
drama.title)
67                         putString("EXTRA_PHOTO",
drama.photo)
68                         putString("EXTRA_PLOT",
drama.plot)
69                         putString("EXTRA_YEAR",
drama.year)
70                         putString("EXTRA_CAST",
drama.cast)
71                     }

```

```

72         }
73     parentFragmentManager.beginTransaction()
74     .replace(R.id.frame_container, detailFragment)
75     .addToBackStack(null)
76     .commit()
77     }
78 }
79 }
80 override fun onDestroyView() {
81     super.onDestroyView()
82     _binding = null
83 }
84 }

```

Tabel 3 Source Code Jawaban HomeFragment.kt

4. DetailFragment

```

1 package com.example.viewmodel
2
3 import android.os.Bundle
4 import androidx.fragment.app.Fragment
5 import android.view.LayoutInflater
6 import android.view.View
7 import android.view.ViewGroup
8 import com.bumptech.glide.Glide
9 import
10 com.example.viewmodel.databinding.FragmentDetail
11 Binding
12
13 class DetailFragment : Fragment() {
14     private var _binding: FragmentDetailBinding?
15     = null
16     private val binding get() = _binding!!
17
18     override fun onCreateView(
19         inflater: LayoutInflater, container:
20         ViewGroup?,
21         savedInstanceState: Bundle?,
22         ): View {
23         _binding =
24         FragmentDetailBinding.inflate(inflater,
25         container, false)
26
27         val title =
28         arguments?.getString("EXTRA_TITLE")
29         val photo =
30         arguments?.getString("EXTRA_PHOTO")

```

```

24         val plot =
arguments?.getString("EXTRA_PLOT")
        val year =
25 arguments?.getString("EXTRA_YEAR")
        val cast =
26 arguments?.getString("EXTRA_CAST")
27
28         android.util.Log.d("DetailFragment",
"Data diterima: $title, $year, $cast")
29
30         binding.tvName.text = title
31         binding.tvPlot.text = plot
32         binding.tvYear.text = "Year: $year"
33         binding.tvCast.text = "Cast: $cast"
34         photo?.let {
35             Glide.with(requireContext())
36                 .load(it)
37                 .into(binding.imgItemPhoto)
38         }
39         binding.buttonKembali.setOnClickListener
{
40             parentFragmentManager.popBackStack()
41         }
42         return binding.root
43     }
44     override fun onDestroyView() {
45         super.onDestroyView()
46         _binding = null
47     }
48 }

```

Tabel 4 Source Code Jawaban DetailFragment.kt

5. DramaAdapter.kt

```

1 package com.example.viewmodel
2
3 import android.view.LayoutInflater
4 import android.view.View
5 import android.view.ViewGroup
6 import android.widget.Button
7 import android.widget.ImageView
8 import android.widget.TextView
9 import androidx.recyclerview.widget.RecyclerView
10 import com.bumptech.glide.Glide
11 import com.example.viewmodel.R
12
13 class DramaAdapter(
14     private var listDrama: List<Drama>,

```

```

15     private val onWikiClick: (String) -> Unit,
16     private val onDetailClick: (Drama) -> Unit
17 ) :
    RecyclerView.Adapter<DramaAdapter.ViewHolder>()
    {
18
19         class ViewHolder(itemView: View) :
    RecyclerView.ViewHolder(itemView) {
20             val imgPhoto: ImageView =
    itemView.findViewById(R.id.img_item_photo)
21             val tvTitle: TextView =
    itemView.findViewById(R.id.tv_item_name)
22             val tvPlot: TextView =
    itemView.findViewById(R.id.tv_item_plot)
                val btnWiki: Button =
23 itemView.findViewById(R.id.btn_wiki)
                val btnDetail: Button =
24 itemView.findViewById(R.id.button_detail)
                }
25
26         override fun onCreateViewHolder(parent:
    ViewGroup, viewType: Int): ViewHolder {
27             val view: View =
28 LayoutInflater.from(parent.context).inflate(R.la
29 yout.item_list, parent, false)
                return ViewHolder(view)
30         }
31
32         override fun getItemCount(): Int =
33 listDrama.size
34
35         override fun onBindViewHolder(holder:
    ViewHolder, position: Int) {
                val drama = listDrama[position]
36
37             android.util.Log.d("DramaAdapter", "Data
38 masuk ke list: ${drama.title}")
39
                holder.tvTitle.text = drama.title
40             holder.tvPlot.text = "Plot:
41 ${drama.plot}"
42
                Glide.with(holder.itemView.context)
43                     .load(drama.photo)
44                     .into(holder.imgPhoto)
45
                holder.btnWiki.setOnClickListener {
46                     android.util.Log.d("DramaAdapter",
47 "Tombol Wiki ditekan untuk: ${drama.title}")
48

```

	onWikiClick(drama.link)
49	}
50	
51	holder.btnDetail.setOnClickListener {
52	android.util.Log.d("DramaAdapter",
53	"Tombol Detail ditekan untuk: \${drama.title}")
	onDetailClick(drama)
54	}
55	}
56	fun updateData(newList: List<Drama>) {
57	listDrama = newList
58	notifyDataSetChanged()
59	}
60	}
61	fun updateData(dramas: List<Drama>) {
62	}
63	

Tabel 5 Source Code Jawaban DramaAdapter.kt

6. DramaViewModel

1	package com.example.viewmodel
2	
3	import android.content.res.Resources
4	import androidx.lifecycle.ViewModel
5	import androidx.lifecycle.viewModelScope
6	import kotlinx.coroutines.flow.MutableSharedFlow
7	import kotlinx.coroutines.flow.MutableStateFlow
8	import kotlinx.coroutines.flow.SharedFlow
9	import kotlinx.coroutines.flow.StateFlow
10	import kotlinx.coroutines.launch
11	
12	class DramaViewModel(private val resources:
	Resources) : ViewModel() {
13	
14	private val _dramaList =
	MutableStateFlow<List<Drama>>(emptyList())
15	val dramaList: StateFlow<List<Drama>> get() =
	_dramaList
16	
17	private val _eventClick =
	MutableSharedFlow<Drama>()
18	val eventClick: SharedFlow<Drama> get() =
	_eventClick
19	
20	init {
21	loadDramas()
22	}

```

23     private fun getDramaFromResources():
List<Drama> {
24         val dataTitle =
resources.getStringArray(R.array.data_name)
25         val dataLink =
resources.getStringArray(R.array.data_link)
26         val dataPhoto =
resources.getStringArray(R.array.data_photo)
27         val dataPlot =
resources.getStringArray(R.array.data_plot)
28         val dataYear =
resources.getStringArray(R.array.data_year)
29         val dataCast =
resources.getStringArray(R.array.data_cast)
30
31         return dataTitle.indices.map { i ->
Drama(
32             title = dataTitle[i],
33             link = dataLink[i],
34             photo = dataPhoto[i],
35             plot = dataPlot[i],
36             year = dataYear[i],
37             cast = dataCast[i]
38         )
39     }
40 }
41
42 private fun loadDramas() {
43     viewModelScope.launch {
44         _dramaList.value =
getDramaFromResources()
45     }
46 }
47 fun onDramaClicked(drama: Drama) {
48     viewModelScope.launch {
49         _eventClick.emit(drama)
50     }
51 }
52 }

```

Tabel 6 Source Code Jawaban DramaViewModel.kt

7. DramaViewModelFactory.kt

```
1 package com.example.viewmodel
2
3 import android.content.res.Resources
4 import androidx.lifecycle.ViewModel
5 import androidx.lifecycle.ViewModelProvider
6
7 class DramaViewModelFactory(private val
resources: Resources) :
ViewModelProvider.Factory {
8     override fun <T : ViewModel>
create(modelClass: Class<T>): T {
9         if
(modelClass.isAssignableFrom(DramaViewModel::cla
ss.java)) {
10             @Suppress("UNCHECKED_CAST")
11             return DramaViewModel(resources) as
T
12         }
13         throw IllegalArgumentException("Unknown
ViewModel class")
14     }
15 }
```

Tabel 7 Source Code Jawaban DramaViewModelFactory.kt

8. Activity_main.xml

```
1 <androidx.constraintlayout.widget.ConstraintLayo
ut
2 xmlns:android="http://schemas.android.com/apk/re
s/android"
3
4 xmlns:app="http://schemas.android.com/apk/res-
auto"
5 xmlns:tools="http://schemas.android.com/tools"
6     android:layout_width="match_parent"
7     android:layout_height="match_parent"
8     tools:context="com.example.viewmodel.MainActivit
y">
9     <FrameLayout
10         android:id="@+id/frame_container"
11         android:layout_width="match_parent"
12         android:layout_height="match_parent" />
13
14 </androidx.constraintlayout.widget.ConstraintLay
out>
```

Tabel 8 Source Code Jawaban Activity_Main.xml

9. fragment_home.xml

1	<androidx.constraintlayout.widget.ConstraintLayout
2	xmlns:android="http://schemas.android.com/apk/res/android"
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	xmlns:tools="http://schemas.android.com/tools"
5	android:layout_width="match_parent"
6	android:layout_height="match_parent"
7	tools:context="com.example.viewmodel.HomeFragment">
8	
9	<TextView
10	android:id="@+id/tvHeader"
11	android:layout_width="0dp"
12	android:layout_height="wrap_content"
13	android:text="@string/header"
14	android:textSize="20sp"
15	android:textStyle="bold"
16	android:background="#872852"
17	android:textColor="@android:color/white"
18	android:gravity="center"
19	android:padding="16dp"
20	app:layout_constraintTop_toTopOf="parent"
21	app:layout_constraintStart_toStartOf="parent"
22	app:layout_constraintEnd_toEndOf="parent" />
23	
24	<androidx.recyclerview.widget.RecyclerView
25	android:id="@+id/rv_Drama"
26	android:layout_width="0dp"
27	android:layout_height="0dp"
28	android:layout_margin="15dp"
29	
30	app:layout_constraintTop_toBottomOf="@id/tvHeader"
31	app:layout_constraintBottom_toBottomOf="parent"
	app:layout_constraintStart_toStartOf="parent"
32	app:layout_constraintEnd_toEndOf="parent" />
33	</androidx.constraintlayout.widget.ConstraintLayout>

Tabel 9 Source Code Jawaban fragment_home.kt

10. fragment_detail.xml

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.constraintlayout.widget.ConstraintLa
3	yout
4	xmlns:android="http://schemas.android.com/apk/
5	res/android"
6	xmlns:app="http://schemas.android.com/apk/res-
7	auto"
8	xmlns:tools="http://schemas.android.com/tools"
9	android:layout_width="match_parent"
10	android:layout_height="match_parent"
11	android:background="@drawable/background_image
12	"
13	tools:context="com.example.viewmodel.DetailFra
14	gment">
15	
16	<ScrollView
17	android:id="@+id/scrollView"
18	android:layout_width="0dp"
19	android:layout_height="0dp"
20	android:fillViewport="true"
21	
22	app:layout_constraintTop_toTopOf="parent"
23	app:layout_constraintBottom_toBottomOf="parent
24	"
25	app:layout_constraintStart_toStartOf="parent"
26	app:layout_constraintEnd_toEndOf="parent">
27	
28	<androidx.constraintlayout.widget.ConstraintLa
29	yout
30	android:layout_width="match_parent"
31	android:layout_height="wrap_content"
32	android:padding="16dp">
33	
34	<ImageView
35	android:id="@+id/img_item_photo"
36	android:layout_width="0dp"
37	android:layout_height="458dp"
38	android:scaleType="centerCrop"
39	app:layout_constraintTop_toTopOf="parent"
40	app:layout_constraintStart_toStartOf="parent"
41	app:layout_constraintEnd_toEndOf="parent" />

```

42         android:layout_marginTop="16dp"
43 app:layout_constraintTop_toBottomOf="@id/img_i
44 tem_photo"
45 app:layout_constraintStart_toStartOf="parent"
46 app:layout_constraintEnd_toEndOf="parent"
47         android:text="Drama Title" />
48
49         <TextView
50             android:id="@+id/tv_year"
51             android:layout_width="wrap_content"
52             android:layout_height="wrap_content"
53             android:text="Year: "
54             android:textSize="16sp"
55             android:layout_marginTop="8dp"
56 app:layout_constraintTop_toBottomOf="@id/tv_na
57 me"
58 app:layout_constraintStart_toStartOf="parent"
59 app:layout_constraintEnd_toEndOf="parent" />
60
61         <TextView
62             android:id="@+id/tv_cast"
63             android:layout_width="0dp"
64             android:layout_height="wrap_content"
65             android:text="Cast: "
66             android:textSize="16sp"
67             android:layout_marginTop="8dp"
68             android:textAlignment="center"
69 app:layout_constraintTop_toBottomOf="@id/tv_ye
70 ar"
71 app:layout_constraintStart_toStartOf="parent"
72 app:layout_constraintEnd_toEndOf="parent" />
73
74         <TextView
75             android:id="@+id/tv_plot"
76             android:layout_width="0dp"
77             android:layout_height="wrap_content"
78             android:layout_marginTop="8dp"
79             android:textAlignment="center"
80             android:textSize="16sp"
81             android:text="Plot: "
82 app:layout_constraintTop_toBottomOf="@id/tv_ca
83 st"
84 app:layout_constraintStart_toStartOf="parent"
85 app:layout_constraintEnd_toEndOf="parent" />
86 </androidx.constraintlayout.widget.ConstraintL
87 ayout>
88 </ScrollView>
89
90 <TextView

```

85	android:id="@+id/button_kembali"
86	android:layout_width="match_parent"
87	android:layout_height="wrap_content"
88	android:layout_margin="16dp"
89	android:background="#d1cfe2"
90	android:elevation="10dp"
91	android:text="Kembali"
92	android:textAllCaps="false"
93	android:textColor="@color/black"
94	android:textSize="20sp"
95	android:gravity="center"
96	android:padding="16dp"
97	app:layout_constraintBottom_toBottomOf="parent"
98	app:layout_constraintStart_toStartOf="parent"
99	app:layout_constraintEnd_toEndOf="parent" />
100	</androidx.constraintlayout.widget.ConstraintLayout>
	>

Tabel 10 Source Code Jawaban fragment_detail.kt

11. item_list.xml

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.cardview.widget.CardView
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	xmlns:app="http://schemas.android.com/apk/res-auto"
5	android:layout_width="match_parent"
6	android:layout_height="wrap_content"
7	xmlns:tools="http://schemas.android.com/tools"
8	android:layout_margin="8dp"
9	app:cardCornerRadius="8dp"
10	app:cardElevation="4dp">
11	<androidx.constraintlayout.widget.ConstraintLayout
12	android:layout_width="match_parent"
13	android:layout_height="wrap_content"
14	android:padding="8dp">
15	<ImageView
16	android:id="@+id/img_item_photo"
17	android:layout_width="70dp"
18	android:layout_height="100dp"
19	

```

20         android:scaleType="centerCrop"
21 app:layout_constraintStart_toStartOf="parent"
22 app:layout_constraintTop_toTopOf="parent"
23 app:layout_constraintBottom_toBottomOf="parent"
24
25         <TextView
26             android:id="@+id/tv_item_name"
27             android:layout_width="0dp"
28             android:layout_height="wrap_content"
29             android:textSize="16sp"
30             android:textStyle="bold"
31 app:layout_constraintStart_toEndOf="@id/img_item
32 _photo"
33 app:layout_constraintTop_toTopOf="parent"
34 app:layout_constraintEnd_toEndOf="parent"
35             android:layout_marginStart="12dp"
36             tools:text="True Beauty" />
37
38         <TextView
39             android:id="@+id/tv_item_plot"
40             android:layout_width="0dp"
41             android:layout_height="wrap_content"
42             android:layout_marginTop="4dp"
43             android:textSize="13sp"
44 app:layout_constraintTop_toBottomOf="@id/tv_item
45 _name"
46 app:layout_constraintStart_toStartOf="@id/tv_ite
47 m_name"
48 app:layout_constraintEnd_toEndOf="parent"
49             tools:text="Plot: " />
50
51         <Button
52             android:id="@+id/btn_wiki"
53             android:layout_width="wrap_content"
54             android:layout_height="wrap_content"
55             android:layout_marginStart="24dp"
56             android:layout_marginTop="8dp"
57             android:backgroundTint="#ffc5d3"
58             android:text="IMDB"
59 app:layout_constraintStart_toStartOf="@id/tv_ite
60 m_plot"
61 app:layout_constraintTop_toBottomOf="@id/tv_item
62 _plot" />
63
64         <Button
65             android:id="@+id/button_detail"
66             android:layout_width="wrap_content"
67             android:layout_height="wrap_content"
68             android:layout_marginStart="8dp"

```

64	android:backgroundTint="#ffc5d3"
65	android:text="Detail"
66	app:layout_constraintStart_toEndOf="@id/btn_wiki
67	"
	app:layout_constraintTop_toTopOf="@id/btn_wiki"
68	/>
	</androidx.constraintlayout.widget.ConstraintLayout>
69	out>
	</androidx.cardview.widget.CardView>
70	

Tabel 11 Source Code Jawaban item_list.kt

12. string.xml

1	<resources>
2	<string name="app_name">modul3</string>
3	<string name="btn_detail">Detail</string>
4	<string name="btn_wiki">Wiki</string>
5	
6	<string-array name="data_name">
7	<item>True Beauty</item>
8	<item>Business Proposal</item>
9	<item>King the Land</item>
10	<item>Welcome to Samdalri</item>
11	<item>Love Next Door</item>
12	<item>Family by Choice</item>
13	<item>Lovely Runner</item>
14	<item>Melo Movie</item>
15	</string-array>
16	
17	<string-array name="data_photo">
18	<item>https://upload.wikimedia.org/wikipedia/id/6/6b/True_Beauty_main_poster.jpg</item>
19	<item>https://upload.wikimedia.org/wikipedia/id/1/19/A_Business_Proposal.jpg</item>
20	<item>https://upload.wikimedia.org/wikipedia/id/d/da/King_the_Land.png</item>
21	<item>https://upload.wikimedia.org/wikipedia/id/5/54/Welcome_to_Samdal-ri.jpg</item>
22	<item>https://upload.wikimedia.org/wikipedia/id/a/a6/Love_Next_Door_poster.png</item>
23	<item>https://upload.wikimedia.org/wikipedia/id/a/a6/Family_by_Choice_poster.png</item>
24	<item>https://upload.wikimedia.org/wikipedia/id/6/67/Lovely_Runner.png</item>
25	<item>https://upload.wikimedia.org/wikipedia/id/

26	7/7a/Melo_Movie_poster.png</item>
27	</string-array>
28	
29	<string-array name="data_link">
30	<item>https://www.netflix.com/id/title/81410834</item>
31	<item>https://www.netflix.com/id-en/title/81509440</item>
32	<item>https://www.netflix.com/id/title/81671440</item>
33	<item>https://www.netflix.com/id/title/81697983</item>
34	<item>https://www.netflix.com/id-en/title/81739044</item>
35	<item>https://www.viu.com/ott/id/id/vod/2435994/Family-By-Choice</item>
36	<item>https://www.netflix.com/id-en/title/81914565</item>
37	<item>https://www.netflix.com/id-en/title/81720593</item>
38	</string-array>
39	
40	<string-array name="data_plot">
41	<item>True Beauty is a high school drama about a girl who becomes a beauty through makeup and faces challenges when her secret is revealed.</item>
42	<item>Business Proposal tells the story of a woman who participates in a business proposal pretending to be her friend, only to find out the man is her company s CEO</item>
43	<item>King the Land is about a CEO who tries to open a new hotel and a woman who works there while struggling with her own love story.</item>
44	<item>Welcome to Samdalri is about a group of individuals who meet in a small village and form bonds through humorous and touching moments.</item>
45	<item>Love Next Door is a romantic drama revolving around a couple living next door and their developing relationship.</item>
46	<item>Family by Choice is a heartwarming drama about a group of people who form a family despite not being related by blood.</item>
47	<item>Lovely Runner follows a runner who is trying to make it in the world of competitive sports while balancing her personal life.</item>

48	<code><item>Melo Movie is a romantic comedy about a woman trying to find love while struggling with her career in the film industry.</item></code>
49	<code></string-array></code>
50	
51	<code><string-array name="data_year"></code>
52	<code><item>2020</item></code>
53	<code><item>2022</item></code>
54	<code><item>2023</item></code>
55	<code><item>2023</item></code>
56	<code><item>2024</item></code>
57	<code><item>2024</item></code>
58	<code><item>2024</item></code>
59	<code><item>2025</item></code>
60	<code></string-array></code>
61	
62	<code><string-array name="data_cast"></code>
63	<code><item>Moon Ga-young, Cha Eun-woo, Hwang In-yeop</item></code>
64	<code><item>Kim Se-jeong, Ahn Hyo-seop, Kim Min Kyu, Seol In Ah</item></code>
65	<code><item>YoonA, Lee Jun-ho</item></code>
66	<code><item>Ji Chang-wook, Shin Hye-sun</item></code>
67	<code><item>Jung Hae In, Jung So Min, Kim Ji Eun, Yun Ji On</item></code>
68	<code><item>Hwang In Yeop, Jung Chae Yeon, Bae Hyun Sung</item></code>
69	<code><item>Byeon Woo-seok, Kim Hye-yeon</item></code>
70	<code><item>Choi Woo Shik, Park Bo Young, Lee Jun Young, Jeon So Nee</item></code>
71	<code></string-array></code>
72	
73	
74	<code><string name="hello_blank_fragment">Hello blank fragment</string></code>
75	<code></resources></code>

Tabel 12 Source Code Jawaban string.kt

13. build.gradle.kts (module:app)

1	<code>plugins {</code>
2	<code>alias(libs.plugins.android.application)</code>
3	<code>alias(libs.plugins.kotlin.android)</code>
4	<code>id("kotlin-parcelize")</code>
5	<code>id("org.jetbrains.kotlin.kapt")</code>
6	<code>}</code>
7	

```

8  android {
9      namespace = "com.example.viewmodel"
10     compileSdk = 35
11
12     defaultConfig {
13         applicationId = "com.example.viewmodel"
14         minSdk = 30
15         targetSdk = 35
16         versionCode = 1
17         versionName = "1.0"
18
19         testInstrumentationRunner =
20         "androidx.test.runner.AndroidJUnitRunner"
21     }
22
23     buildTypes {
24         release {
25             isMinifyEnabled = false
26             proguardFiles(
27                 getDefaultProguardFile("proguard-android-
28                 optimize.txt"),
29                 "proguard-rules.pro"
30             )
31         }
32     }
33     compileOptions {
34         sourceCompatibility =
35         JavaVersion.VERSION_11
36         targetCompatibility =
37         JavaVersion.VERSION_11
38     }
39     kotlinOptions {
40         jvmTarget = "11"
41     }
42     buildFeatures {
43         viewBinding = true
44     }
45 }
46
47 dependencies {
48     implementation(libs.androidx.core.ktx)
49     implementation(libs.androidx.appcompat)
50     implementation(libs.material)
51     implementation(libs.androidx.activity)
52     implementation("androidx.recyclerview:recyclerview:1.3.1")

```

	implementation
51	("androidx.constraintlayout:constraintlayout:2.1.4")
52	testImplementation(libs.junit)
53	androidTestImplementation(libs.androidx.junit)
54	androidTestImplementation(libs.androidx.espresso
	.core)
55	implementation("com.github.bumptech.glide:glide:4.15.1")
56	kapt("com.github.bumptech.glide:compiler:4.15.1")
57	implementation("androidx.lifecycle:lifecycle-viewmodel-ktx:2.7.0")
58	implementation("org.jetbrains.kotlinx:kotlinx-coroutines-core:1.7.3")
59	implementation("org.jetbrains.kotlinx:kotlinx-coroutines-android:1.7.3")
60	implementation("androidx.fragment:fragment-ktx:1.6.0")
	}

Tabel 13 Source Code Jawaban build.gradle.kts

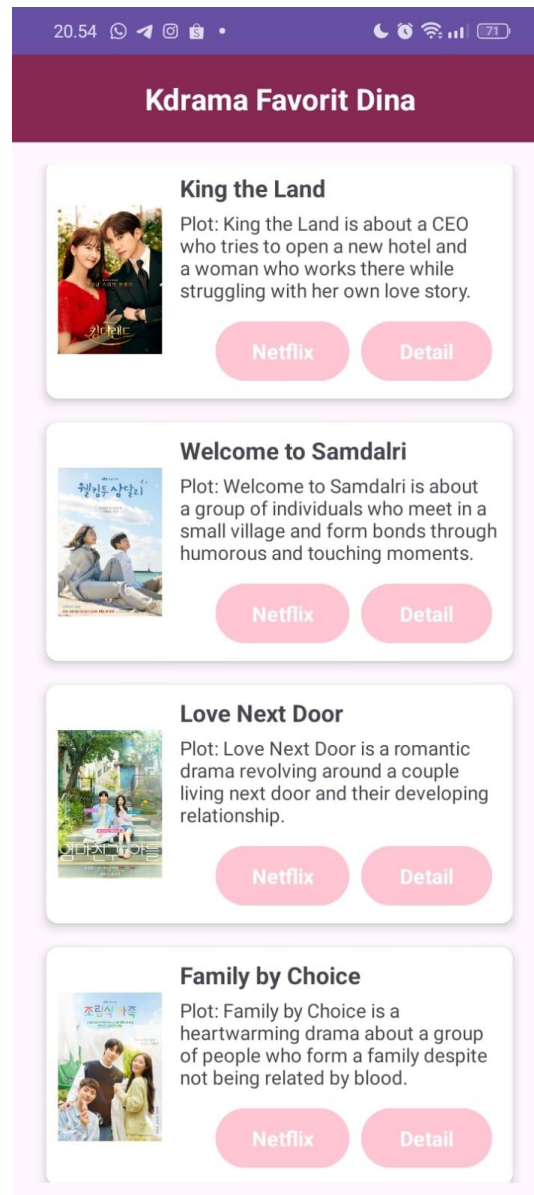
14. AndroidManifest.kt

1	<?xml version="1.0" encoding="utf-8"?>
2	<manifest
	xmlns:android="http://schemas.android.com/apk/res/android"
3	xmlns:tools="http://schemas.android.com/tools">
4	
5	<uses-permission
6	android:name="android.permission.INTERNET"/>
	<application
7	android:allowBackup="true"
8	android:dataExtractionRules="@xml/data_extraction_rules"
9	android:fullBackupContent="@xml/backup_rules"
10	android:icon="@mipmap/ic_launcher"
11	android:label="@string/app_name"
12	android:roundIcon="@mipmap/ic_launcher_round"
13	android:supportsRtl="true"
14	android:theme="@style/Theme.ViewModel"
15	tools:targetApi="31">
16	<activity
17	android:name="com.example.viewmodel.MainActivity"
18	android:exported="true">
19	<intent-filter>

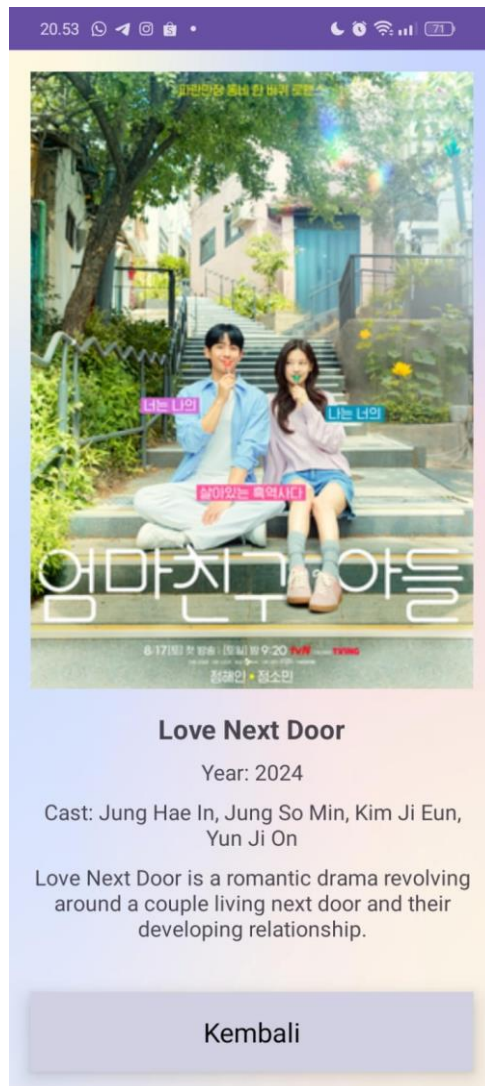
20	<action
21	android:name="android.intent.action.MAIN" />
	<category
22	android:name="android.intent.category.LAUNCHER"
	/>
23	</intent-filter>
24	</activity>
25	</application>
26	</manifest>

Tabel 14 Source Code Jawaban AndoidManifest.kt

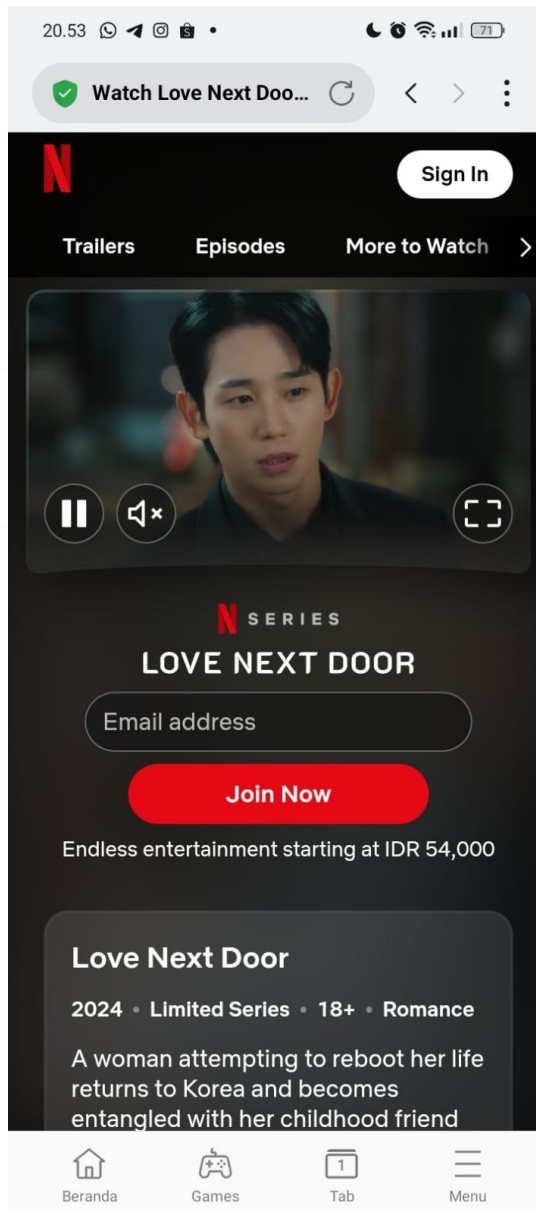
B. Output Program



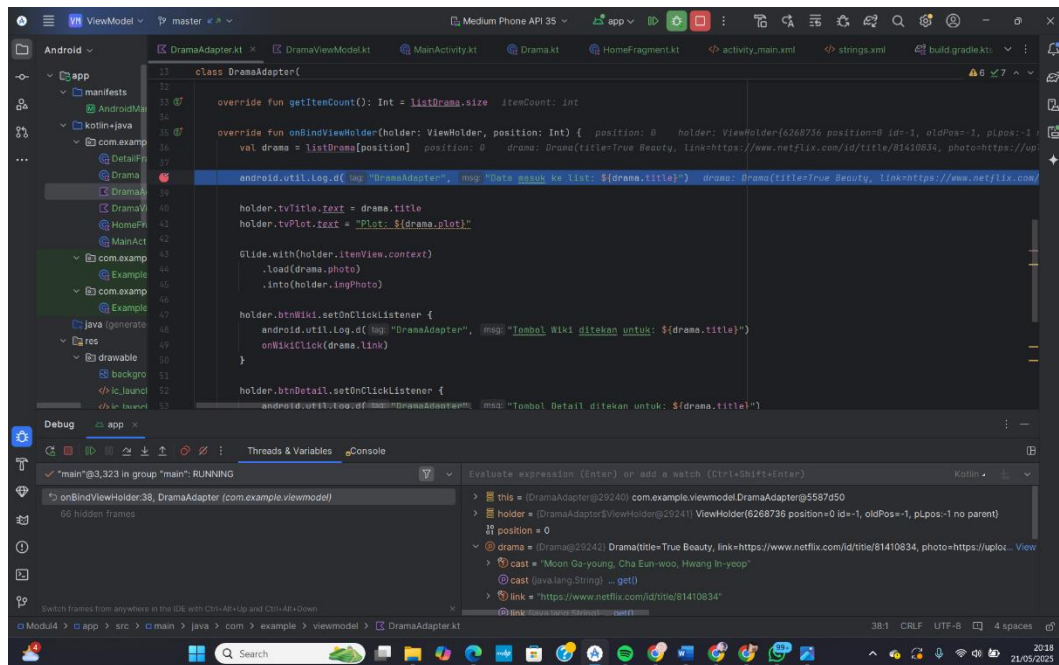
Gambar 1 Halaman Home



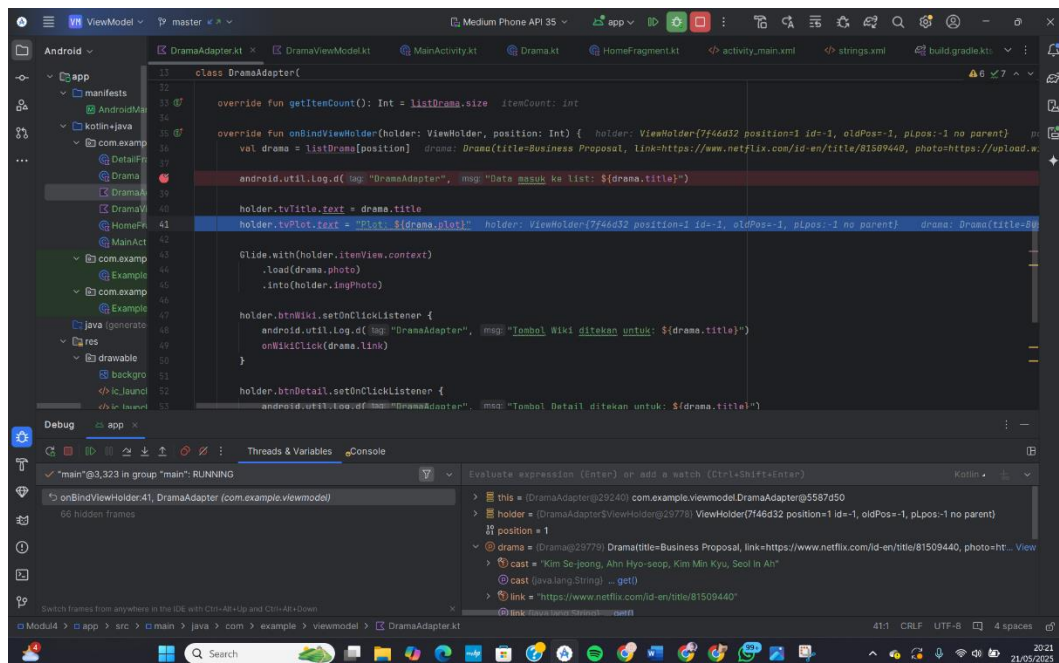
Gambar 2 Halaman Detail



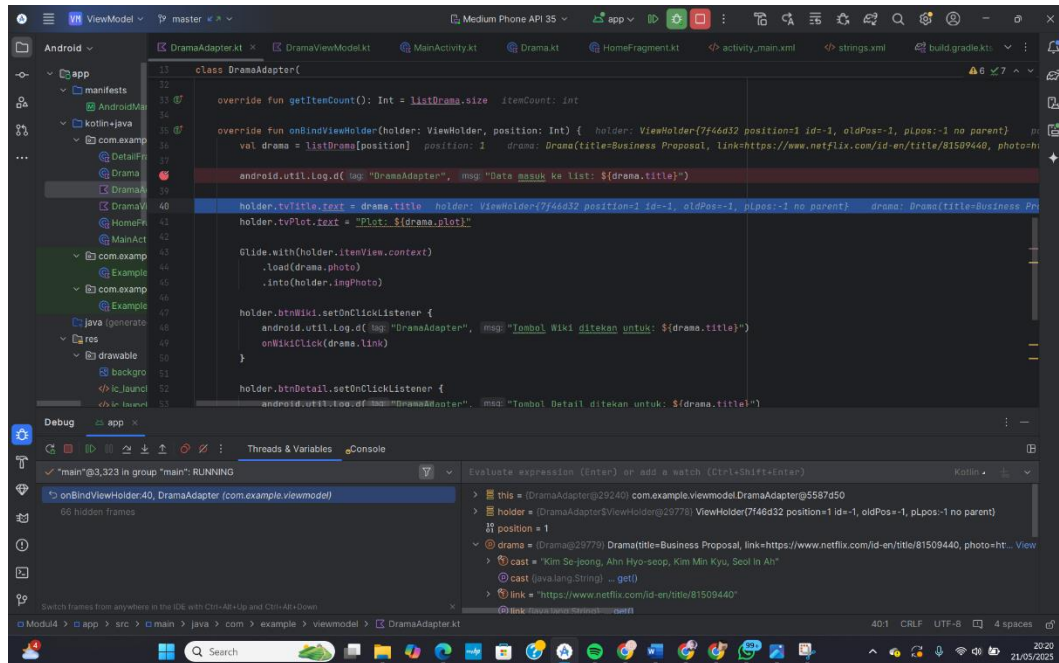
Gambar 3 Link Netflix



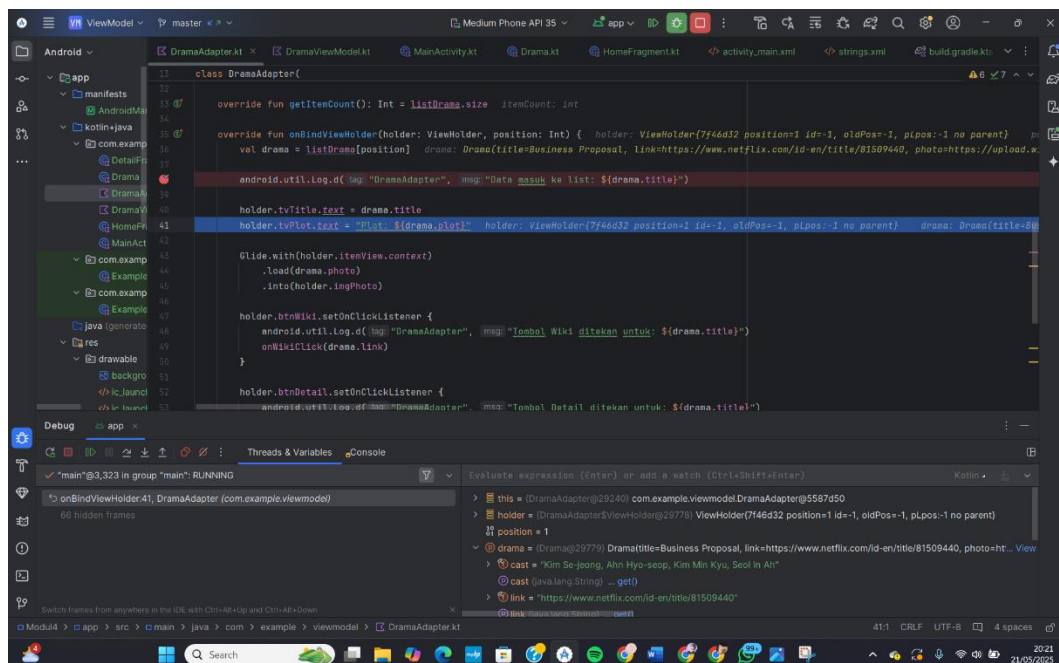
Gambar 4 Penggunaan Debugger



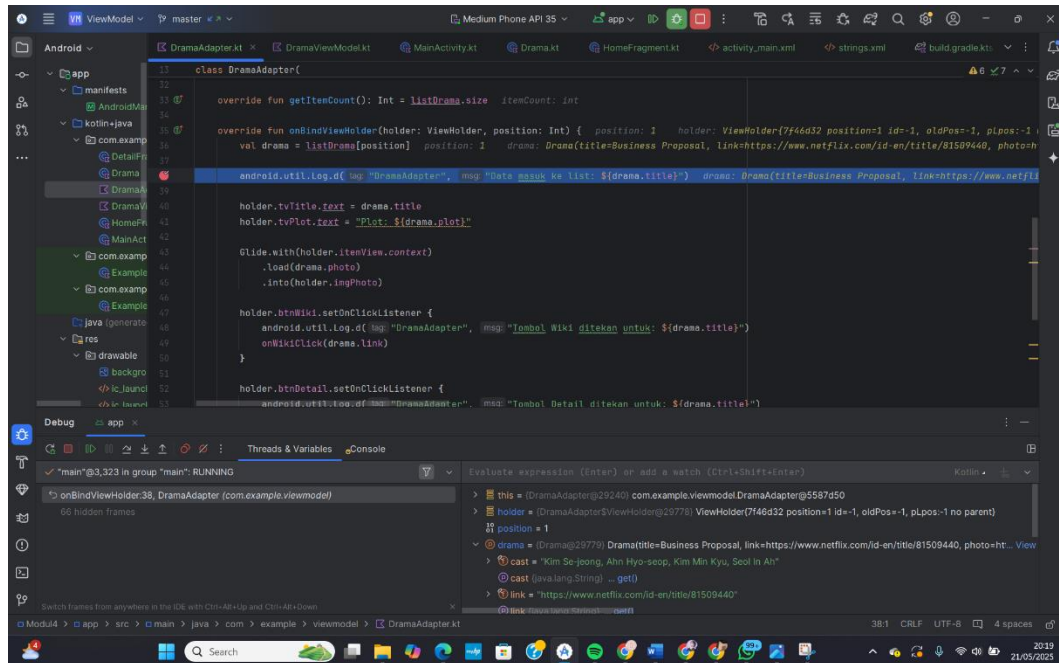
Gambar 5 Penggunaan Fitur f7 (Step Into) pada Debug



Gambar 6 Penggunaan Fitur f8 (Step Over) pada Debug



Gambar 7 Penggunaan Fitur Shift+F8 (Step Out) pada Debug



Gambar 8 Penggunaan Fitur f9 (Resume Program) pada Debug

C. Pembahasan

1. MainActivity.kt

Pada baris 1 package `com.example.viewmodel` menunjukkan bahwa file Kotlin ini berada di dalam paket bernama `viewmodel` di bawah domain `com.example`. Baris 3–5 merupakan impor agar file ini bisa menggunakan `AppCompatActivity` dan resource dari `R`. Baris 7 mendefinisikan kelas `MainActivity` yang merupakan activity utama aplikasi ini. Baris 8 memanggil fungsi `onCreate()` yang akan dijalankan saat activity dibuat. Baris 10 menjalankan `setContentView(R.layout.activity_main)` untuk menampilkan layout `activity_main` sebagai antarmuka pengguna. Baris 12 membuat objek `FragmentManager` untuk menangani manajemen fragmen di activity ini. Baris 13 membuat instance dari `HomeFragment`. Baris 14–15 memeriksa apakah fragmen dengan tag `HomeFragment` sudah ada sebelumnya, dan jika belum maka fragmen baru akan ditambahkan ke dalam `frame_container` menggunakan `beginTransaction()` lalu disimpan dengan `commit()`.

2. Drama.kt

Pada bagian awal, kode `package com.example.recyclerview` digunakan untuk mengelompokkan file ini ke dalam paket `recyclerview` agar lebih terorganisir dalam struktur proyek. Baris `import android.os.Parcelable` dan `import kotlinx.parcelize.Parcelize` memungkinkan kelas `Drama` dapat dikirim antar komponen Android dengan cara efisien menggunakan `parcel`. Anotasi `@Parcelize` digunakan supaya proses implementasi `Parcelable` tidak perlu dilakukan secara manual, cukup dengan menandai data class tersebut. Kelas `Drama` bertipe data `class` yang menyimpan informasi seperti `title`, `link`, `photo`, `plot`, `year`, dan `cast` yang masing-masing bertipe `String`. Kelas ini mengimplementasikan `Parcelable`, sehingga objeknya dapat dipaketkan dan ditransfer antar `Activity` atau `Fragment`.

3. HomeFragment.kt

Pada baris 1 `package com.example.viewmodel` menunjukkan bahwa file Kotlin ini berada dalam paket bernama `viewmodel` di bawah domain `com.example`. Baris 3–15 adalah bagian `import` yang memungkinkan kelas ini menggunakan berbagai komponen seperti `Intent`, `Uri`, `Fragment`, `ViewModel`, `RecyclerView`, dan `coroutine` dari `lifecycle`. Baris 17 `class HomeFragment : Fragment()` mendefinisikan bahwa `HomeFragment` merupakan subclass dari `Fragment`. Baris 19–20 mendeklarasikan variabel binding dengan `FragmentHomeBinding` untuk mengakses elemen UI dari layout `fragment_home` tanpa perlu menggunakan `findViewById` secara manual. Baris 22–24 mendeklarasikan `viewModel` menggunakan `delegate by viewModels` dan memanfaatkan `DramaViewModelFactory` yang mengambil resource dari `context`. Baris 26–32 mengimplementasikan fungsi `onCreateView` yang meng-inflate layout XML menggunakan binding dan mengembalikan root view sebagai tampilan `fragment`. Di dalam `onViewCreated`, `RecyclerView` disiapkan dengan `LinearLayoutManager` agar daftar drama

ditampilkan secara vertikal, dan `setHasFixedSize(true)` digunakan untuk meningkatkan performa. Sebuah instance `DramaAdapter` dibuat dengan parameter kosong untuk `listDrama`, dan dua aksi klik: `onWikiClick` membuka link dengan `Intent`, sementara `onDetailClick` memicu event dari `ViewModel` saat item diklik. Adapter ini kemudian diset ke `RecyclerView`. `Coroutine` pertama di dalam `lifecycleScope.launch` berfungsi untuk mengamati aliran data `dramaList` dari `ViewModel` dan memperbarui isi adapter setiap kali data berubah. `Coroutine` kedua mengamati `eventClick`, yaitu event klik item, dan jika ada data yang diterima, akan dibuat instance `DetailFragment` dengan mengisi `Bundle` berisi informasi drama yang diklik. Fragment tersebut kemudian ditampilkan menggunakan transaksi fragment dan ditambahkan ke back stack. Terakhir, fungsi `onDestroyView` digunakan untuk menghapus nilai binding menjadi null agar mencegah memory leak setelah view dihancurkan.

4. `DetailFragment.kt`

Pada baris 1 `package com.example.viewmodel` menunjukkan bahwa file Kotlin ini berada dalam paket bernama `viewModel` di bawah domain `com.example`. Baris 3–9 adalah bagian import yang memungkinkan penggunaan berbagai komponen Android seperti `Fragment`, `Bundle`, `View`, `ViewGroup`, serta library eksternal `Glide` untuk pemrosesan gambar. Baris 11 `class DetailFragment : Fragment()` mendefinisikan bahwa `DetailFragment` adalah turunan dari `Fragment`. Baris 13–14 mendeklarasikan binding menggunakan `FragmentDetailBinding` agar bisa mengakses elemen UI dari layout `fragment_detail` secara langsung. Baris 16 sampai akhir di dalam fungsi `onCreateView`, layout di-inflate menggunakan binding lalu mengambil data dari `arguments` dengan key seperti `EXTRA_TITLE`, `EXTRA_PHOTO`, `EXTRA_PLOT`, `EXTRA_YEAR`, dan `EXTRA_CAST`, kemudian diset ke `TextView` dan `ImageView` yang sesuai. `Glide` digunakan untuk memuat gambar dari URL ke dalam `imgItemPhoto`

secara efisien. Tombol kembali disiapkan untuk memanggil `popBackStack` yang akan menutup fragment ini dan kembali ke sebelumnya. Fungsi ini mengembalikan tampilan dari root binding. Terakhir, fungsi `onDestroyView` digunakan untuk menghapus binding agar tidak terjadi memory leak saat fragment dihancurkan.

5. DramaAdapter

Pada baris 1 `package com.example.viewmodel` menunjukkan bahwa file Kotlin ini berada dalam paket bernama `viewModel` di bawah domain `com.example`. Baris 3–10 adalah bagian import yang memungkinkan penggunaan komponen Android seperti `View`, `RecyclerView`, `TextView`, `ImageView`, `Button`, dan library `Glide` untuk memuat gambar dari URL. Baris 12 `class DramaAdapter` mendefinisikan adapter untuk `RecyclerView` yang menerima data berupa `List<Drama>` serta dua lambda, yaitu `onWikiClick` dan `onDetailClick`, yang masing-masing akan dijalankan ketika tombol Wiki dan tombol Detail ditekan. Baris 19–25 mendefinisikan `ViewHolder` sebagai class dalam yang bertanggung jawab menginisialisasi dan mereferensikan elemen UI dalam item, seperti gambar, teks judul, teks plot, dan dua tombol dari file layout `item_list.xml`. Baris 27 `onCreateViewHolder` bertugas untuk meng-inflate layout `item_list` dan mengembalikannya sebagai objek `ViewHolder`. Baris 33 `getItemCount` akan mengembalikan jumlah data yang ada di dalam `listDrama` untuk menentukan berapa banyak item yang akan ditampilkan. Baris 35 `onBindViewHolder` menghubungkan data drama berdasarkan posisi ke komponen UI dalam item, memuat gambar menggunakan `Glide`, menampilkan teks judul dan plot, serta memberikan aksi ketika tombol Wiki ditekan (memanggil `onWikiClick`) dan tombol Detail ditekan (memanggil `onDetailClick`). Baris 57–61 adalah fungsi `updateData` yang digunakan untuk memperbarui isi daftar drama dengan data baru dan memanggil `notifyDataSetChanged()`

agar tampilan RecyclerView ikut diperbarui. Baris 62–63 adalah deklarasi fungsi `updateData` kosong yang tidak memiliki isi dan bisa dihapus karena tidak digunakan.

6. DramaViewModel

Pada baris 1 `package com.example.viewmodel` menunjukkan bahwa file Kotlin ini berada dalam paket bernama `viewModel` di bawah domain `com.example`. Baris 3–9 adalah bagian import yang diperlukan untuk menggunakan kelas `ViewModel`, `coroutine`, serta `StateFlow` dan `SharedFlow` dari Kotlin. Baris 11 `class DramaViewModel` mendefinisikan kelas `ViewModel` yang bertugas menyediakan dan mengelola data list drama, dengan menerima parameter `Resources` untuk mengambil data dari file arrays di `resources`. Baris 14–15 mendeklarasikan `_dramaList` sebagai `MutableStateFlow` yang berisi list dari objek `Drama`, lalu membungkusnya menjadi `StateFlow` agar hanya dapat diamati dari luar tanpa bisa dimodifikasi langsung. Baris 17–18 mendeklarasikan `_eventClick` sebagai `MutableSharedFlow` untuk menangani event klik item yang akan dibagikan ke observer, lalu dibungkus sebagai `SharedFlow`. Baris 20–22 memanggil fungsi `loadDramas()` saat `ViewModel` pertama kali dibuat untuk memuat data dari `resources`. Fungsi `getDramaFromResources()` pada baris 23–41 mengambil data dari array string seperti judul, link, foto, plot, tahun, dan pemain dari `resources`, lalu menggabungkannya menjadi list berisi objek `Drama`. Baris 42–46 merupakan implementasi fungsi `loadDramas()` yang menjalankan `coroutine` menggunakan `viewModelScope` untuk mengisi nilai `_dramaList` dengan hasil dari `getDramaFromResources()`. Terakhir, baris 47–52 adalah fungsi `onDramaClicked()` yang juga dijalankan dalam `coroutine` untuk mengirimkan objek `Drama` yang diklik melalui `_eventClick` agar bisa diproses di fragment.

7. DramaViewModelFactory.kt

Pada baris 1 `package com.example.viewmodel` menunjukkan bahwa file Kotlin ini berada dalam paket bernama `viewModel` di bawah domain `com.example`. Baris 3–5 adalah bagian `import` yang digunakan untuk mengakses `Resources`, `ViewModel`, dan `ViewModelProvider` dari `Android`. Baris 7 `class DramaViewModelFactory` mendefinisikan sebuah kelas `factory` yang mewarisi `ViewModelProvider.Factory`, dan bertugas untuk membuat instance dari `DramaViewModel` dengan membawa parameter `resources`. Baris 8–12 mengimplementasikan fungsi `create` yang akan dipanggil saat `ViewModel` diminta, lalu memeriksa apakah class yang diminta adalah `DramaViewModel`. Jika benar, maka akan dikembalikan instance `DramaViewModel` dengan `cast` ke tipe generic `T`, sedangkan jika bukan, maka akan dilempar `IllegalArgumentException` untuk menangani kasus class yang tidak dikenali.

8. Activity_Main.xml

Baris 1 `<androidx.constraintlayout.widget.ConstraintLayout>` adalah layout utama yang digunakan sebagai wadah seluruh elemen UI di halaman ini. Baris 2–4 adalah deklarasi namespace XML yang memungkinkan penggunaan atribut `Android`, `app` (untuk `ConstraintLayout`), dan `tools`. Baris 5–7 mendefinisikan ukuran layout agar memenuhi seluruh layar serta menetapkan konteks `tools` untuk keperluan preview di `Android Studio`. Baris 9–12 `<FrameLayout>` dengan id `frame_container` digunakan sebagai kontainer fragmen yang akan diganti-ganti secara dinamis, misalnya saat navigasi antar halaman. Terakhir, baris 14 menutup tag `ConstraintLayout` sebagai akhir dari struktur layout ini.

9. fragment_home.xml

Baris 1 `<androidx.constraintlayout.widget.ConstraintLayout>` digunakan sebagai layout utama yang memungkinkan penempatan elemen UI secara fleksibel dengan constraint. Baris 2-4 adalah deklarasi namespace XML standar agar atribut seperti `android:`, `app:`, dan `tools:` dapat digunakan dengan benar. Baris 5-7 mengatur layout agar mengisi seluruh layar dan menyetel konteks preview di Android Studio ke `HomeFragment`. Baris 9-22 `<TextView>` dengan id `tvHeader` menampilkan teks dari resource `@string/header` sebagai judul halaman dengan ukuran 20sp, teks putih tebal, latar belakang ungu gelap, dan posisi berada di bagian atas layout serta disejajarkan secara horizontal di tengah. Baris 24-32 `<androidx.recyclerview.widget.RecyclerView>` dengan id `rv_Drama` digunakan untuk menampilkan daftar drama dalam bentuk list yang bisa discroll secara vertikal, ditempatkan tepat di bawah `tvHeader` dengan margin di sekelilingnya, dan memenuhi sisa ruang layar. Terakhir, baris 33 menutup tag `ConstraintLayout` sebagai akhir dari struktur layout ini.

10. fragment_detail.xml

Baris 1 `<androidx.constraintlayout.widget.ConstraintLayout>` digunakan sebagai layout utama yang memungkinkan penempatan elemen UI secara fleksibel dengan constraint. Baris 2–5 adalah deklarasi namespace XML standar agar atribut seperti `android:`, `app:`, dan `tools:` dapat digunakan dengan benar dalam layout. Baris 6–9 mengatur ukuran layout agar memenuhi layar penuh, memberikan latar belakang gambar dari drawable, serta mengatur konteks pratinjau layout ke `DetailFragment`. Baris 10–19 adalah elemen `ScrollView` dengan `fillViewport=true` agar konten dapat discroll jika melebihi tinggi layar, dan dikunci dengan constraint ke seluruh sisi parent. Baris 21–81 adalah nested `ConstraintLayout` di dalam `ScrollView` untuk menampung konten vertikal seperti gambar dan teks dengan pengaturan padding 16dp. Baris 26–33 `<ImageView>` dengan id `img_item_photo` digunakan untuk menampilkan

gambar poster drama dengan ukuran tetap 458dp dan skala `centerCrop`, serta diletakkan di bagian atas layout. Baris 35–45 `<TextView>` dengan id `tv_name` menampilkan judul drama dengan ukuran teks 20sp dan gaya tebal, diletakkan di bawah gambar dan diposisikan di tengah. Baris 47–56 `<TextView>` dengan id `tv_year` menampilkan informasi tahun rilis drama dengan ukuran 16sp, margin atas 8dp, dan posisi di bawah judul. Baris 58–68 `<TextView>` dengan id `tv_cast` berisi daftar pemain drama, dengan lebar penuh, alignment tengah, dan margin atas 8dp, diposisikan di bawah tahun. Baris 70–680 `<TextView>` dengan id `tv_plot` menampilkan sinopsis atau alur cerita dengan alignment tengah dan ukuran teks 16sp, diletakkan di bawah informasi cast. Baris 84-99 `<TextView>` dengan id `button_kembali` berfungsi sebagai tombol kembali dengan latar warna ungu muda, teks rata tengah, ukuran 20sp, dan dikunci ke bagian bawah layout agar selalu tampil tetap meskipun konten di-scroll.

11. item_list.xml

Baris 1 `<androidx.cardview.widget.CardView>` digunakan sebagai wadah utama dengan sudut membulat dan efek elevasi agar tampilan setiap item terlihat seperti kartu. Baris 3-5 adalah deklarasi namespace XML standar yang memungkinkan penggunaan atribut seperti `android:`, `app:`, dan `tools:`. Baris 5-7 mengatur lebar kartu agar memenuhi parent, tinggi menyesuaikan isi, dengan margin 8dp serta pengaturan radius sudut 8dp dan elevasi 4dp agar tampak sedikit melayang. Baris 12-15 `<androidx.constraintlayout.widget.ConstraintLayout>` digunakan sebagai layout dalam kartu untuk menyusun elemen-elemen UI secara terstruktur. Baris 17-24 `<ImageView>` dengan id `img_item_photo` digunakan untuk menampilkan gambar poster drama berukuran 70x100dp dengan skala `centerCrop`, diposisikan di sisi kiri dan vertikal tengah dalam kartu. Baris 26-36 `<TextView>` dengan id `tv_item_name` menampilkan judul drama dengan teks tebal berukuran 16sp, diletakkan di samping gambar dengan margin awal 12dp dan disejajarkan ke kanan. Baris 38-47 `<TextView>` dengan id `tv_item_plot` menampilkan deskripsi singkat atau

plot drama di bawah judul, dengan ukuran teks lebih kecil 13sp dan sedikit margin atas. Baris 49-58 `<Button>` dengan id `btn_wiki` berfungsi untuk membuka tautan Netflix, diberi teks “Netflix”, latar belakang berwarna pink muda, dan diletakkan di bawah plot dengan margin awal 24dp. Baris 60-68 `<Button>` dengan id `button_detail` digunakan untuk menampilkan detail drama, posisinya tepat di samping kanan tombol Netflix dan tetap sejajar secara vertikal. Terakhir, baris 71 menutup tag `ConstraintLayout` dan baris 72 menutup tag `CardView` sebagai akhir dari struktur layout ini

12. string.xml

Baris 1 `<resources>` merupakan elemen root pada file XML ini yang berisi semua data string dan array yang digunakan dalam aplikasi. Baris 2-4 mendefinisikan string statis seperti nama aplikasi dan label pada tombol untuk detail serta wiki. Baris 6-15 `<string-array name="data_name">` menyimpan daftar judul drama Korea yang akan ditampilkan dalam `RecyclerView`. Baris 17-26 `<string-array name="data_photo">` menyimpan daftar URL gambar dari internet yang digunakan sebagai foto untuk setiap judul drama. Karena aplikasi menggunakan library `Glide` untuk memuat gambar. Baris 28-37 `<string-array name="data_link">` berisi link menuju halaman resmi masing-masing drama, kebanyakan dari Netflix dan satu dari Viu. Baris 39-48 `<string-array name="data_plot">` berisi deskripsi singkat atau sinopsis dari setiap drama, menjelaskan tema dan alur utama cerita. Baris 50-59 `<string-array name="data_year">` menyimpan tahun rilis masing-masing drama agar bisa ditampilkan pada tampilan detail. Baris 61-70 `<string-array name="data_cast">` memuat daftar pemeran utama dari masing-masing drama. Baris 73 dan 74 menambahkan string statis untuk keperluan default fragment dan judul header halaman. Terakhir, baris 75 menutup tag `<resources>` sebagai akhir dari seluruh deklarasi string dan array yang ada di file ini.

13. Build.gradle.kts

Pada gradle ini saya menambahkan baris 3 yaitu `id("kotlin-parcelize")` digunakan untuk mengaktifkan plugin Kotlin Parcelize yang memungkinkan penggunaan anotasi `@Parcelize` pada kelas data agar objeknya bisa dengan mudah dikirim antar komponen Android seperti Activity atau Fragment melalui Intent atau Bundle. Baris 4 Plugin `id("org.jetbrains.kotlin.kapt")` diaktifkan agar proyek dapat menggunakan KAPT (Kotlin Annotation Processing Tool), yang diperlukan ketika menggunakan library yang mengandalkan annotation processor Glide. Lalu pada `dependencies` menambahkan `implementation("androidx.recyclerview:recyclerview:1.3.1")` yang digunakan untuk menyertakan library RecyclerView versi 1.3.1 untuk menampilkan daftar item secara efisien dan fleksibel dalam bentuk list atau grid yang dapat discroll. Dan `implementation("androidx.constraintlayout:constraintlayout:2.1.4")` digunakan untuk memasukkan library ConstraintLayout versi 2.1.4 yang memberikan kontrol layout yang lebih fleksibel dan efisien dengan sistem constraint antar elemen UI. Baris 54 `implementation("com.github.bumptech.glide:glide:4.15.1")` berfungsi untuk mengimpor library utama Glide versi 4.15.1, yang memungkinkan aplikasi Android memuat gambar dari berbagai sumber, termasuk URL di internet. baris 55 `kapt("com.github.bumptech.glide:compiler:4.15.1")` digunakan untuk mengaktifkan dukungan annotation processor yang disediakan oleh Glide. Baris 56 `implementation("androidx.lifecycle:lifecycle-viewmodel-ktx:2.7.0")` digunakan untuk menyertakan library Lifecycle ViewModel versi KTX, yang mendukung penggunaan arsitektur MVVM pada Android. Dengan ekstensi `ktx`, ViewModel dapat digunakan secara lebih idiomatik dengan Kotlin, termasuk dukungan terhadap coroutine melalui `viewModelScope` agar tugas asinkron dapat dijalankan dengan aman dan efisien tanpa menyebabkan memory leak. Baris 57 `implementation("org.jetbrains.kotlinx:kotlinx-coroutines`

core:1.7.3") digunakan untuk menambahkan library Kotlin Coroutines Core versi 1.7.3, yang merupakan pustaka inti untuk pemrograman asynchronous dan concurrency di Kotlin. Dengan ini, kita dapat menggunakan fungsi suspend, launch, async, dan coroutine flow untuk menjalankan operasi non-blocking secara efisien. Baris 58 `implementation("org.jetbrains.kotlinx:kotlinx-coroutines-android:1.7.3")` digunakan untuk menambahkan dukungan coroutine pada platform Android, yang memungkinkan eksekusi coroutine pada `Dispatchers.Main` atau thread utama. Library ini mempermudah pengelolaan tugas latar belakang yang perlu memperbarui UI setelah selesai. Baris 59 `implementation("androidx.fragment:fragment-ktx:1.6.0")` digunakan untuk menyertakan library ekstensi Kotlin (KTX) pada Fragment, yang menyediakan API berbasis Kotlin agar lebih ringkas dan efisien. Dengan ini, kita bisa menggunakan fitur seperti `findNavController()`, `lifecycleScope`, dan `setFragmentResultListener()` secara langsung dan lebih idiomatik.

14. AndroidManifest.xml

Pada file ini, saya menambahkan baris `<uses-permission android:name="android.permission.INTERNET"/>` yang merupakan deklarasi izin (permission) untuk memberi tahu sistem Android bahwa aplikasi membutuhkan akses ke jaringan internet. Izin ini sangat penting terutama jika aplikasi memuat konten dari sumber eksternal seperti gambar dari URL menggunakan library Glide. Selain itu, saya juga menyertakan deklarasi `<activity android:name="com.example.viewmodel.MainActivity">` yang dilengkapi dengan `<intent-filter>` untuk menetapkan MainActivity sebagai titik masuk utama aplikasi. Baris `<action android:name="android.intent.action.MAIN" />` menandakan bahwa aktivitas ini akan dijalankan pertama kali saat aplikasi dibuka, sedangkan `<category android:name="android.intent.category.LAUNCHER" />` memastikan bahwa aktivitas ini akan muncul di daftar aplikasi (launcher).

pada perangkat Android, sehingga pengguna dapat meluncurkan aplikasi melalui ikon di layar utama. Atribut tambahan seperti `android:theme`, `android:icon`, dan `android:label` digunakan untuk menentukan tampilan dan nama aplikasi di perangkat. Sementara itu, `android:allowBackup`, `android:fullBackupContent`, dan `android:dataExtractionRules` mengatur perilaku pencadangan data aplikasi, dan `android:supportsRtl="true"` memungkinkan aplikasi mendukung tata letak kanan ke kiri (RTL) untuk bahasa tertentu.

15. Pembahasan Debugging

Debugger adalah alat penting dalam pengembangan aplikasi Android yang memungkinkan pemeriksaan eksekusi kode secara mendetail saat aplikasi berjalan. Fungsi utamanya adalah membantu menemukan dan memperbaiki bug atau kesalahan dalam kode dengan cara menghentikan sementara jalannya program pada titik tertentu (breakpoint), memeriksa nilai variabel, dan melacak alur eksekusi kode secara real-time. Dengan debugger, pengembang dapat memahami bagaimana aplikasi bekerja secara internal dan mengidentifikasi penyebab masalah dengan lebih cepat dan akurat.

Cara Menggunakan Debugger

1. Pasang Breakpoint:

Tentukan titik di kode (breakpoint) tempat yang ingin program berhenti sementara. Caranya, klik di sebelah kiri baris kode yang ingin dicek.

2. Jalankan Program dalam Mode Debug:

Tekan tombol Debug 'app' di Android Studio. Program akan berjalan seperti biasa sampai menemukan breakpoint, lalu berhenti di sana.

3. Periksa Nilai Variabel dan Alur Eksekusi:

Setelah program berhenti, lihat nilai variabel, array, atau objek di panel Variables. Juga bisa lihat urutan pemanggilan fungsi di panel Frames.

4. Menggunakan Fitur Step (Step Into, Step Over, Step Out):

Untuk menelusuri kode lebih detail, gunakan fitur step yang disediakan oleh debugger

Fungsi Step Into, Step Over, Step Out, dan Resume Program di Debug

a. Step Into

Fitur ini digunakan untuk masuk ke dalam detail fungsi atau metode yang sedang dipanggil pada baris kode saat ini. Jika pada baris tersebut terdapat pemanggilan fungsi, Step Into akan membawa proses debug masuk ke dalam fungsi tersebut dan menjalankan kode di dalamnya secara langkah demi langkah. Fitur ini sangat berguna untuk memahami secara mendalam apa yang terjadi di dalam fungsi tersebut, bukan hanya melihat hasil akhirnya saja.

b. Step Over

Step Over menjalankan baris kode saat ini secara utuh, termasuk pemanggilan fungsi, tanpa masuk ke dalam fungsi tersebut. Setelah baris tersebut selesai dieksekusi, proses debug akan berhenti di baris berikutnya. Fitur ini cocok digunakan ketika ingin mengetahui hasil dari fungsi tanpa perlu menelusuri langkah-langkah di dalamnya secara detail.

c. Step Out

Step Out digunakan ketika proses debug sudah berada di dalam sebuah fungsi, namun ingin keluar dan kembali ke baris kode setelah pemanggilan fungsi tersebut di bagian pemanggil. Debugger akan menjalankan sisa kode dalam fungsi tersebut sampai selesai, kemudian berhenti di baris setelah pemanggilan fungsi. Fitur ini membantu mempercepat proses debugging dengan keluar dari fungsi yang sudah cukup diperiksa.

d. Resume Program

Resume Program berfungsi untuk melanjutkan eksekusi program secara normal hingga menemukan breakpoint berikutnya atau sampai program selesai berjalan. Fitur ini memungkinkan proses debug berjalan tanpa berhenti di setiap baris kode, sehingga debugging dapat dilanjutkan dengan lebih cepat setelah pemeriksaan tertentu selesai.

Referensi: <https://developer.android.com/codelabs/basic-android-kotlin-compose-intro-debugger?hl=id#0>

https://google-developer-training.github.io/android-developer-fundamentals-course-concepts/idn/Unit%201/31_c_the_android_studio_debugger.html

D. Tautan Git

https://github.com/dinzti/Pemrograman_Mobile

SOAL 2

Jelaskan Application class dalam arsitektur aplikasi Android dan fungsinya

A. Jawaban

Application class di Android adalah kelas dasar yang menjadi pondasi utama dari sebuah aplikasi Android. Berfungsi sebagai titik awal (entry point) aplikasi dan bertanggung jawab untuk mengelola status global aplikasi, serta menjalankan inisialisasi penting sebelum komponen lain seperti Activity, Service, atau BroadcastReceiver dibuat. Application class dibuat secara otomatis oleh sistem Android saat aplikasi diluncurkan, dan tetap aktif sampai aplikasi dihentikan atau dikeluarkan dari memori.

Fungsi Application Class antara lain:

1. Menjaga Status Global

Application class sangat berguna untuk menyimpan data atau status yang ingin diakses secara global oleh seluruh komponen aplikasi. Misalnya, Anda bisa menyimpan objek singleton, konfigurasi global, atau resource yang dibutuhkan di banyak tempat dalam aplikasi.

2. Inisialisasi Awal

Salah satu fungsi utama Application class adalah menjalankan proses inisialisasi yang hanya perlu dilakukan sekali selama siklus hidup aplikasi. Contohnya, inisialisasi library pihak ketiga (seperti database, analytics, atau dependency injection), setup konfigurasi, atau registrasi receiver global. Semua proses ini biasanya diletakkan di dalam metode onCreate() pada subclass Application.

3. Manajemen Resource dan Lifecycle

Application class juga menyediakan beberapa metode lifecycle seperti onCreate(), onConfigurationChanged(), dan onLowMemory(). Dengan override metode-metode ini, developer bisa mengatur perilaku aplikasi saat terjadi perubahan konfigurasi (misal: perubahan orientasi layar) atau ketika sistem mendeteksi memori yang hampir habis.

Contoh sederhana Application Class seperti berikut:

kotlin

```
class MyApplication : Application() {  
    override fun onCreate() {  
        super.onCreate()  
        // Inisialisasi global di sini  
    }  
}
```

Dan AndroidManifest.xml:

xml

```
<application  
    android:name=".MyApplication"  
    ... >  
</application>
```

Application class merupakan pusat pengelolaan status dan inisialisasi global pada aplikasi Android. Sangat penting dalam arsitektur aplikasi modern karena memungkinkan pengaturan resource dan proses yang berlaku untuk seluruh aplikasi, bukan hanya pada satu Activity atau Service saja.

Referensi: <https://www.linkedin.com/pulse/application-class-android-amit-nadiger/>

https://github.com/codepath/android_guides/wiki/Understanding-the-Android-Application-Class